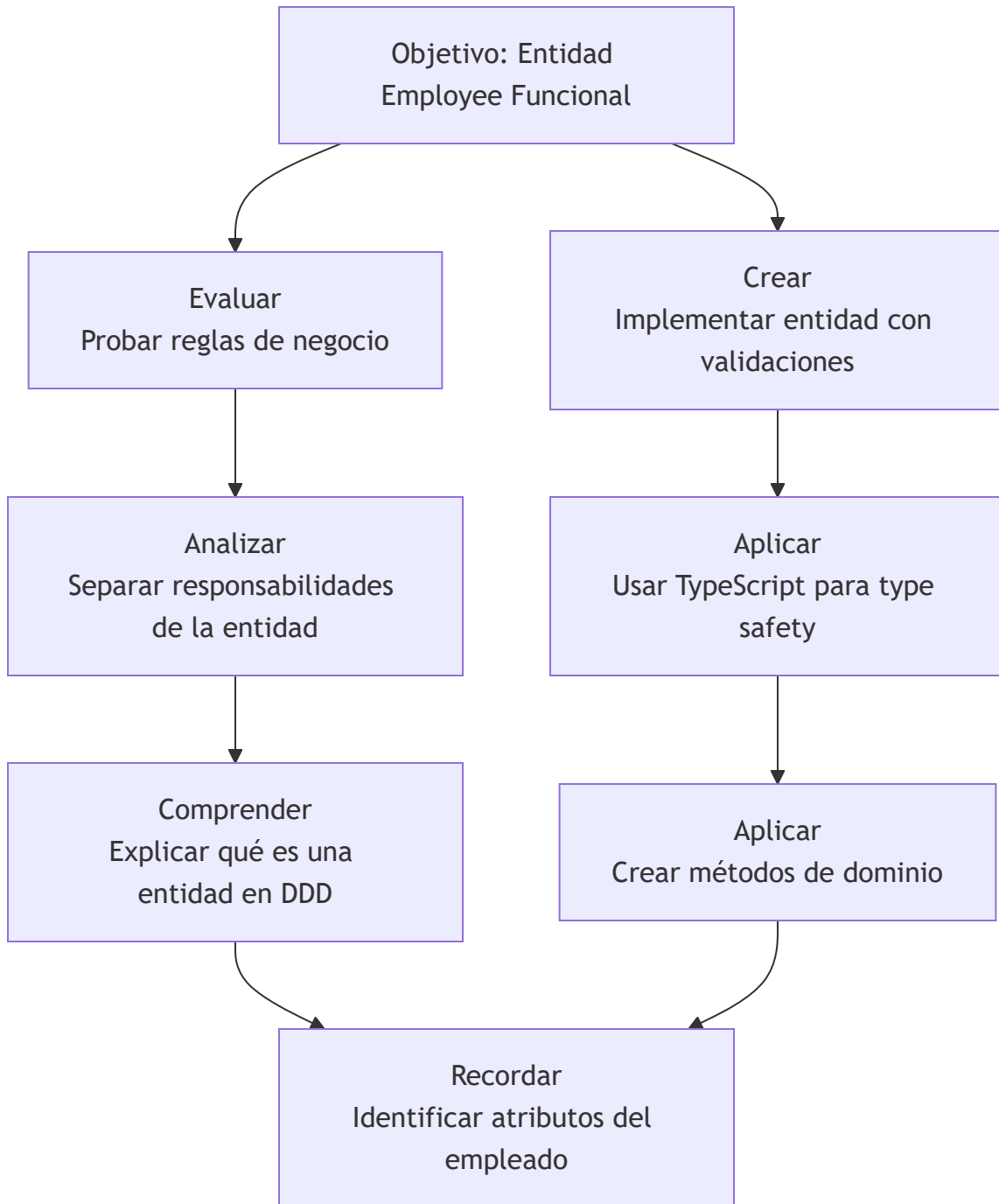


Guía 2: Capa de Dominio - Entidad

Employee

Objetivos de Aprendizaje

Pirámide de Objetivos - Taxonomía de Bloom

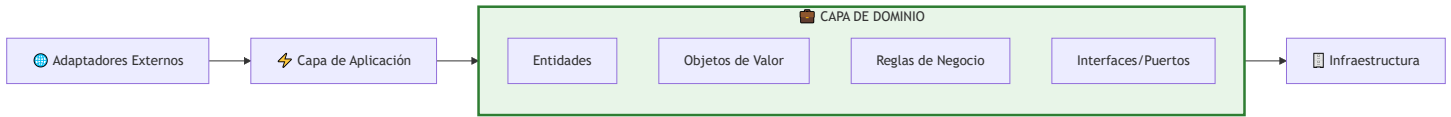


Objetivos Específicos por Verbo

Nivel Cognitivo	Verbo	Objetivo
Recordar	Identificar	Reconocer los atributos básicos de un empleado
Comprender	Explicar	Describir qué es una entidad en Domain-Driven Design
Aplicar	Implementar	Codificar la clase Employee en TypeScript
Aplicar	Validar	Agregar validaciones básicas a los atributos
Analizar	Diseñar	Separar métodos de dominio de métodos de acceso
Evaluar	Probar	Verificar que la entidad funciona correctamente
Crear	Ejecutar	Probar la entidad en consola con datos de prueba

1. ¿Qué es la Capa de Dominio?

Diagrama Conceptual de la Arquitectura Hexagonal



¿Qué va en la Capa de Dominio?

Componente	Descripción	Ejemplo en nuestro proyecto
Entidades	Objetos con identidad única y ciclo de vida	Employee
Reglas de Negocio	Validaciones y lógica específica del dominio	"Salario no puede ser negativo"
Interfaces/Puertos	Contratos que definen cómo interactuar	EmployeeRepository
Excepciones	Errores específicos del dominio	EmployeeNotFoundError

Características clave del Dominio:

- ☒ **Independiente** de frameworks y tecnologías
- ☒ **Contiene** las reglas de negocio fundamentales
- ☒ **Define** el lenguaje del dominio (ubiquitous language)
- ☒ **No tiene** dependencias externas

2. Creando la Entidad Employee

2.1 Estructura de la Entidad

```
// 📁 src/domain/employee.entity.ts

/**
 * 🎯 ENTIDAD EMPLOYEE
 *
 * * Una entidad en Domain-Driven Design es un objeto que:
 * 1. Tiene identidad única (ID)
 * 2. Tiene un ciclo de vida
 * 3. Puede cambiar manteniendo su identidad
 * 4. Contiene reglas de negocio
 *
 * * En nuestro caso, Employee representa a un empleado
 *   con cédula, nombres y salario.
 */

// Interface para los datos de la entidad
export interface EmployeeData {
  id: string;
  cedula: string;
  nombres: string;
  salario: number;
  createdAt: Date;
  updatedAt: Date;
}

export class Employee {
  // Propiedades PRIVADAS (encapsulamiento)
  private id: string;
  private cedula: string;
  private nombres: string;
  private salario: number;
  private createdAt: Date;
  private updatedAt: Date;

  /**
   * Constructor PRIVADO
   * Usamos factory methods para crear instancias
   */
}
```

```

private constructor(data: EmployeeData) {
    this.id = data.id;
    this.cedula = data.cedula;
    this.nombres = data.nombres;
    this.salario = data.salario;
    this.createdAt = data.createdAt;
    this.updatedAt = data.updatedAt;

    // Validar que la entidad sea válida
    this.validate();
}

/**
 * 📄 MÉTODO FACTORY: create()
 * Crea una NUEVA entidad Employee
 * Se usa cuando registramos un nuevo empleado
 */
static create(cedula: string, nombres: string, salario: number): Employee {
    // Generar ID único (simple para este ejemplo)
    const id = 'emp-' + Date.now() + '-' + Math.random().toString(36).substr(2, 9);

    const now = new Date();

    return new Employee({
        id,
        cedula: cedula.trim(),
        nombres: nombres.trim(),
        salario,
        createdAt: now,
        updatedAt: now
    });
}

/**
 * 📄 MÉTODO FACTORY: fromData()
 * Reconstruye una entidad desde datos persistidos
 * Se usa cuando cargamos desde la base de datos
 */
static fromData(data: EmployeeData): Employee {
    return new Employee(data);
}

/**

```

```

*  MÉTODO: validate()
* Valida las reglas de negocio de la entidad
* Se llama automáticamente en el constructor
*/
private validate(): void {
    // Validar cédula (mínimo 10 caracteres)
    if (!this.cedula || this.cedula.length < 10) {
        throw new Error('La cédula debe tener al menos 10 caracteres');
    }

    // Validar nombres (mínimo 2 caracteres)
    if (!this.nombres || this.nombres.length < 2) {
        throw new Error('Los nombres deben tener al menos 2 caracteres');
    }

    // Validar salario (no negativo)
    if (this.salario < 0) {
        throw new Error('El salario no puede ser negativo');
    }

    // Validar fechas
    if (!(this.createdAt instanceof Date) || !(this.updatedAt instanceof Date)) {
        throw new Error('Las fechas deben ser instancias de Date');
    }
}

/**
*  MÉTODOS DE DOMINIO (comportamiento)
* Estos métodos representan ACCIONES que puede realizar un empleado
* o que se pueden realizar SOBRE un empleado
*/

/**
* Cambiar los nombres del empleado
*/
changeNombres(nuevosNombres: string): void {
    if (!nuevosNombres || nuevosNombres.trim().length < 2) {
        throw new Error('Los nuevos nombres deben tener al menos 2 caracteres');
    }

    this.nombres = nuevosNombres.trim();
    this.updatedAt = new Date(); // Actualizar timestamp
}

```

```

/**
 * Cambiar el salario del empleado
 */
changeSalario(nuevoSalario: number): void {
  if (nuevoSalario < 0) {
    throw new Error('El salario no puede ser negativo');
  }

  this.salario = nuevoSalario;
  this.updatedAt = new Date(); // Actualizar timestamp
}

/**
 * Aumentar salario por porcentaje
 */
increaseSalario(porcentaje: number): void {
  if (porcentaje < 0) {
    throw new Error('El porcentaje no puede ser negativo');
  }

  if (porcentaje > 100) {
    throw new Error('El porcentaje no puede ser mayor a 100');
  }

  const aumento = this.salario * (porcentaje / 100);
  this.salario += aumento;
  this.updatedAt = new Date();
}

/**
 * 📖 GETTERS (métodos de acceso)
 * Proporcionan acceso controlado a las propiedades
 */

getId(): string {
  return this.id;
}

getCedula(): string {
  return this.cedula;
}

```



```

getNombres(): string {
    return this.nombres;
}

getSalario(): number {
    return this.salario;
}

getCreatedAt(): Date {
    return this.createdAt;
}

getUpdatedAt(): Date {
    return this.updatedAt;
}

/**
 * 🔄 MÉTODO: toJSON()
 * Convierte la entidad a un objeto plano
 * Útil para serializar (API, base de datos, etc.)
 */
toJSON(): EmployeeData {
    return {
        id: this.id,
        cedula: this.cedula,
        nombres: this.nombres,
        salario: this.salario,
        createdAt: this.createdAt,
        updatedAt: this.updatedAt
    };
}

/**
 * 🔍 MÉTODO: toString()
 * Representación en string de la entidad
 * Útil para logging y debugging
 */
toString(): string {
    return `Employee[id=${this.id}, cedula=${this.cedula}, nombres=${this.nombres}, salario=${this.salario}, createdAt=${this.createdAt}, updatedAt=${this.updatedAt}]`;
}

```



3. Probando la Entidad en Consola

3.1 Crear Archivo de Prueba

```
// 📁 src/test-employee.ts
import { Employee } from '../domain/employee.entity.js';

console.log('🔧 PROBANDO ENTIDAD EMPLOYEE');
console.log('=====\\n');

try {
  // 📌 PRUEBA 1: Crear un empleado válido
  console.log('📌 Prueba 1: Crear empleado válido');
  const empleado1 = Employee.create('1234567890', 'Juan Pérez', 2500);
  console.log('✅ Empleado creado:', empleado1.toString());
  console.log('  ID:', empleado1.getId());
  console.log('  Cédula:', empleado1.getCedula());
  console.log('  Nombres:', empleado1.getNombres());
  console.log('  Salario:', empleado1.getSalario());
  console.log('  Creado:', empleado1.getCreatedAt().toLocaleString());
  console.log('');

  // 📌 PRUEBA 2: Cambiar nombres
  console.log('📌 Prueba 2: Cambiar nombres');
  empleado1.changeNombres('Juan Carlos Pérez');
  console.log('✅ Nombres cambiados a:', empleado1.getNombres());
  console.log('  Actualizado:', empleado1.getUpdatedAt().toLocaleString());
  console.log('');

  // 📌 PRUEBA 3: Cambiar salario
  console.log('📌 Prueba 3: Cambiar salario');
  empleado1.changeSalario(2800);
  console.log('✅ Salario cambiado a:', empleado1.getSalario());
  console.log('');

  // 📌 PRUEBA 4: Aumentar salario
  console.log('📌 Prueba 4: Aumentar salario 10%');
  empleado1.increaseSalario(10);
  console.log('✅ Salario después de aumento:', empleado1.getSalario().toFixed(2));
  console.log('');

  // 📌 PRUEBA 5: Convertir a JSON
```

```

console.log('🚩 Prueba 5: Convertir a JSON');
const jsonData = empleado1.toJSON();
console.log('✅ Datos JSON:', JSON.stringify(jsonData, null, 2));
console.log('');

// 🚩 PRUEBA 6: Reconstruir desde datos
console.log('🚩 Prueba 6: Reconstruir desde datos');
const empleado2 = Employee.fromData(jsonData);
console.log('✅ Empleado reconstruido:', empleado2.toString());
console.log('');

// 🚩 PRUEBA 7: Crear empleado inválido (debe fallar)
console.log('🚩 Prueba 7: Intentar crear empleado inválido');
try {
  const empleadoInvalido = Employee.create('123', '', -1000);
  console.log('❌ ERROR: Debió fallar pero no falló');
} catch (error: any) {
  console.log('✅ Correctamente falló:', error.message);
}
console.log('');

// 🚩 PRUEBA 8: Cambios inválidos (deben fallar)
console.log('🚩 Prueba 8: Intentar cambios inválidos');
try {
  empleado1.changeSalario(-500);
  console.log('❌ ERROR: Debió fallar pero no falló');
} catch (error: any) {
  console.log('✅ Correctamente falló:', error.message);
}
console.log('');

// 🚩 PRUEBA 9: Aumento inválido (debe fallar)
console.log('🚩 Prueba 9: Intentar aumento inválido');
try {
  empleado1.increaseSalario(-5);
  console.log('❌ ERROR: Debió fallar pero no falló');
} catch (error: any) {
  console.log('✅ Correctamente falló:', error.message);
}

} catch (error: any) {
  console.error('❌ ERROR INESPERADO:', error.message);
  console.error(error.stack);
}

```

```
}
```

```
console.log('\n🚀 PRUEBAS COMPLETADAS');
```

3.2 Agregar Script de Prueba a package.json

```
// 📄 package.json (actualizar la sección scripts)
{
  "scripts": {
    "dev": "nodemon src/app.ts",
    "build": "tsc",
    "start": "node dist/app.js",
    "setup": "powershell -File .\\setup-directories.ps1",
    "verify": "powershell -File .\\verify-structure.ps1",
    "test:entity": "npx ts-node src/test-employee.ts"
  }
}
```

3.3 Ejecutar las Pruebas

```
# Ejecutar desde tu PowerShell:
npm run test:entity // si falla ejecutar lo sgte

# 1. Compilar TypeScript a JavaScript
npx tsc

# 2. Verificar que se crearon los archivos .js
ls dist/domain/employee.entity.js
ls dist/test-employee.js

# 3. Ejecutar el JavaScript compilado
node dist/test-employee.js
```

Salida esperada:

PROBANDO ENTIDAD EMPLOYEE

=====

Prueba 1: Crear empleado válido

 Empleado creado: Employee[id=emp-1234567890-abc123, cedula=1234567890, nombres=Juan Pérez, s
ID: emp-1234567890-abc123
Cédula: 1234567890
Nombres: Juan Pérez
Salario: 2500
Creado: 15/01/2025, 10:30:00

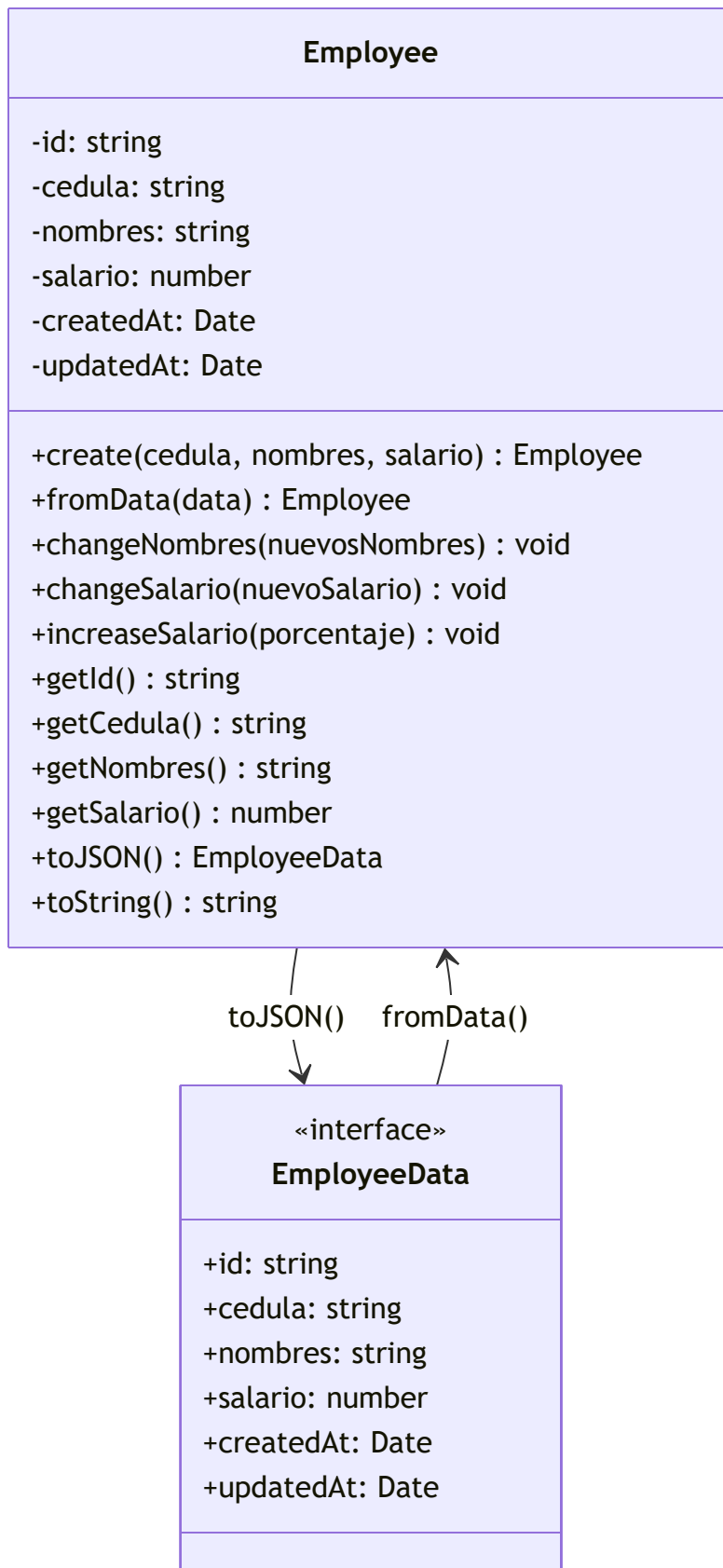
Prueba 2: Cambiar nombres

 Nombres cambiados a: Juan Carlos Pérez
Actualizado: 15/01/2025, 10:30:01

... (resto de las pruebas)

4. Conceptos Clave Explicados

Diagrama de la Entidad Employee



Principios Aplicados en la Entidad

Principio	Cómo se aplica	Beneficio
Encapsulamiento	Propiedades privadas, métodos públicos	Controla el acceso, evita estado inconsistente
Inmutabilidad	ID y createdAt no se pueden cambiar	Garantiza identidad consistente
Validación	validate() en constructor	Garantiza integridad de datos
Factory Methods	create() y fromData()	Control centralizado de creación
Separación de preocupaciones	Métodos de dominio vs getters	Código más claro y mantenible

Reglas de Negocio Implementadas

- 1. **Cédula:** Mínimo 10 caracteres
- 2. **Nombres:** Mínimo 2 caracteres, se triman espacios
- 3. **Salario:** No puede ser negativo
- 4. **Porcentaje de aumento:** Entre 0 y 100%
- 5. **Timestamps:** Se actualizan automáticamente



5. Ejercicios para Estudiantes

Ejercicio 1: Extender la Entidad

```
// Agrega estos métodos a la entidad y prueba:

// 1. Método para obtener salario anual
getSalarioAnual(): number {
  // Tu código aquí
}

// 2. Método para verificar si el empleado gana más de un monto
ganaMasDe(monto: number): boolean {
  // Tu código aquí
}

// 3. Método para formatear nombres en mayúsculas
getNombresEnMayusculas(): string {
  // Tu código aquí
}
```

Ejercicio 2: Agregar Validaciones

```
// Agrega estas validaciones:

// 1. Validar que la cédula solo contenga números
if (!/^d+$/ .test(this.cedula)) {
  throw new Error('La cédula solo puede contener números');
}

// 2. Validar que el salario no exceda un máximo
if (this.salario > 100000) {
  throw new Error('El salario no puede exceder $100,000');
}

// 3. Validar formato de nombres (al menos nombre y apellido)
const partesNombre = this.nombres.split(' ');
if (partesNombre.length < 2) {
  throw new Error('Debe ingresar al menos nombre y apellido');
}
```


Ejercicio 3: Probar Casos Especiales

Crea un archivo `src/test-casos-especiales.ts` y prueba:

```
// 1. Empleado con cédula exactamente de 10 caracteres
// 2. Empleado con nombres con muchos espacios
// 3. Empleado con salario cero
// 4. Intentar cambiar nombres a string vacío
// 5. Intentar aumentar salario 0%
```



6. Checklist de Completitud

Para el Estudiante:

- ☐ Entidad Employee creada en `src/domain/employee.entity.ts`
- ☐ Métodos de dominio implementados (`changeNombres`, `changeSalario`, etc.)
- ☐ Validaciones básicas funcionando
- ☐ Factory methods (`create` y `fromData`) implementados
- ☐ Archivo de prueba `src/test-employee.ts` creado
- ☐ Script `npm run test:entity` agregado a `package.json`
- ☐ Pruebas ejecutadas y funcionando
- ☐ Entendí qué es una entidad en Domain-Driven Design

Para el Profesor:

- ☐ Verificar que la entidad sigue principios de encapsulamiento
- ☐ Revisar que las validaciones son adecuadas
- ☐ Confirmar que los factory methods funcionan correctamente
- ☐ Verificar que el testing cubre casos válidos e inválidos
- ☐ Evaluar comprensión de conceptos de dominio

? Preguntas de Reflexión

1. ¿Por qué el constructor es privado?

Para forzar el uso de factory methods y mantener control sobre la creación.

2. ¿Qué ventaja tiene separar create() de fromData()?

create() es para nuevos empleados, fromData() para reconstruir desde persistencia.

3. ¿Por qué las propiedades son privadas?

Encapsulamiento: controlamos cómo se accede y modifican los datos.

4. ¿Qué pasa si intentamos crear un empleado con salario negativo?

El método validate() lanza un error antes de que se cree la instancia.

5. ¿Por qué actualizamos updatedAt en cada cambio?

Para trackear cuándo fue la última modificación (auditoría).



7. Próximos Pasos

Guía 3 - Interfaz del Repositorio:

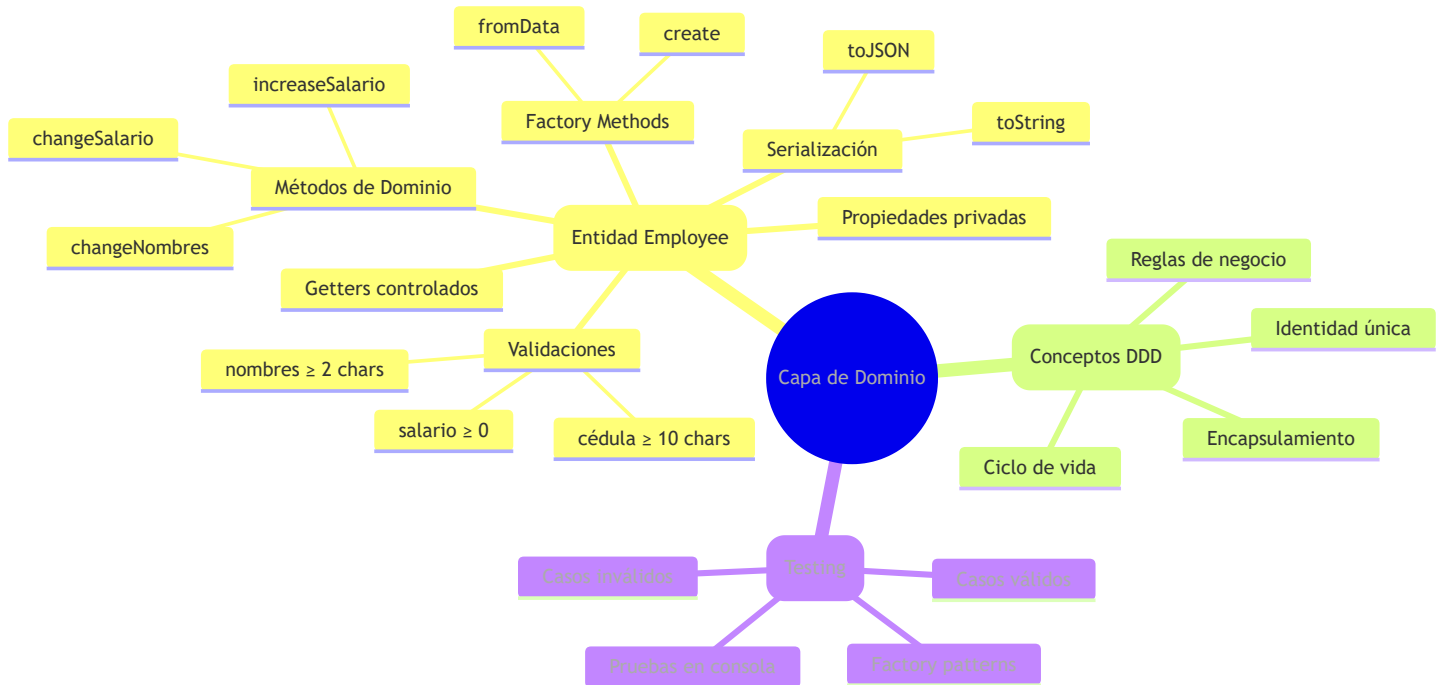
- Crear la interfaz `EmployeeRepository`
- Definir los métodos CRUD como contratos
- Explicar el patrón Puerto/Adaptador

Tarea para estudiantes:

1. Ejecutar `npm run test:entity` y capturar la salida
2. Extender la entidad con un método adicional (ej: calcular bono)
3. Investigar sobre Value Objects en DDD



Resumen de lo Aprendido



✓ Has completado la Guía 2:

- ✓ Entendiste qué es una entidad en Domain-Driven Design
- ✓ Implementaste la entidad Employee con TypeScript
- ✓ Aplicaste principios de encapsulamiento y validación
- ✓ Creaste factory methods para controlar la creación
- ✓ Probaste la entidad exhaustivamente en consola